Практически задачи: SQL DDL/DML и релационен модел

За темата "Бази от данни. Релационен модел на данните" + връзка с нормални форми

Как да решаваш

- Пиши валиден SQL. Приемай PostgreSQL-подобен синтаксис, освен ако задачата не изисква друго.
- За DDL задачите винаги включвай PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE/CHECK, когато логически следват от условието.
- При M:N връзка използвай свързваща таблица с композитен първичен ключ.
- Кратък ключ за проверка има накрая. На изпит е достатъчно да дадеш коректна структура, не непременно същите имена.

Част A - DDL команди

A1. Movie database schema (10 т.)

Проектирайте SQL схема за база данни за филми. Един режисьор има много филми. Един филм може да има много жанрове и един жанр може да принадлежи на много филми.

Създайте таблици directors, movies, genres, movie_genres. Добавете подходящи PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE и CHECK ограничения.

A2. ALTER TABLE и ограничения (8 т.)

Дадена е таблица movies(movie_id, title, release_year, director_id). Напишете DDL команди за:

- добавяне на runtime_min, което трябва да е положително число;
- добавяне на UNIQUE(title, release_year);
- добавяне на FOREIGN KEY director_id към directors(director_id);
- създаване на индекс по director_id.

A3. Students, courses, enrollments (10 т.)

Проектирайте схема за студенти и курсове. Всеки студент може да се запише в много курсове, а всеки курс има много студенти. За всяко записване се пази grade от 2 до 6 и enrollment_date.

Напишете CREATE TABLE командите. Използвайте свързваща таблица enrollments с композитен ключ.

A4. DDL след нормализация (12 т.)

Дадена е ненормализирана таблица:

```
Orders(order_id, customer_id, customer_name, product_id, product_name, quantity)
```

Функционални зависимости: customer_id -> customer_name; product_id -> product_name; order_id, product_id -> quantity. Декомпозирайте до 3НФ и напишете DDL за получените таблици.

A5. ON DELETE поведение (7 т.)

В базата Movies решете какво ON DELETE поведение е подходящо и напишете FK частите:

- ако филм се изтрие, редовете му в movie_genres да се изтрият автоматично;
- ако жанр се използва от филм, да не може да се изтрие;
- ако режисьор се използва от филм, да не може да се изтрие.

Част B - DML команди

B1. INSERT с външни ключове (6 т.)

Напишете INSERT команди за добавяне на режисьор Christopher Nolan, жанр Drama, филм Oppenheimer от 2023 г. и връзка между филма и жанра. Приемете, че използвате ръчно зададени id стойности.

B2. UPDATE и DELETE (6 т.)

Напишете DML команди за: 1) промяна на runtime_min на Oppenheimer на 180; 2) изтриване само на връзката между филм с movie_id = 10 и genre_id = 3 от movie_genres.

Част C - SQL заявки и релационна алгебра

C1. JOIN заявка (7 т.)

Напишете SQL заявка, която извежда title, release_year и full_name на режисьора за всички филми след 2020 г., сортирани по release_year намаляващо.

C2. GROUP BY и HAVING (8 т.)

Напишете SQL заявка, която извежда всеки режисьор и броя на филмите му, но само за режисьори с поне 2 филма.

C3. Релационна алгебра към SQL (8 т.)

Преведете израза в SQL:

```
π title ( σ release_year >= 2020 ( movies ) )
```

C4. Естествено съединение/Join (8 т.)

С таблици movies(movie_id, title, director_id) и directors(director_id, full_name), напишете SQL еквивалент на съединение по director_id и извеждане само на title и full_name.

Част D - Нормални форми + DDL

D1. От FD към таблици (12 т.)

Дадено е R(StudentID, StudentName, CourseID, CourseName, TeacherID, TeacherName, Grade) със зависимости:

```
StudentID -> StudentName
CourseID -> CourseName, TeacherID
TeacherID -> TeacherName
StudentID, CourseID -> Grade
```

1) Намерете основния кандидат-ключ. 2) Посочете аномалиите. 3) Декомпозирайте до 3НФ. 4) Напишете DDL за получените таблици.

D2. BCNF vs 3NF and SQL schema (12 т.)

Дадено е R(A, B, C) с F = { AB -> C, C -> B }. 1) Намерете кандидат-ключовете. 2) Проверете 3НФ и BCNF. 3) Предложете декомпозиция до BCNF. 4) Запишете DDL с абстрактни типове INTEGER за получените релации.

Кратък ключ за проверка

A1 - примерен отговор

```
CREATE TABLE directors (
  director_id INTEGER PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  birth_year INTEGER CHECK (birth_year BETWEEN 1880 AND 2100)
);
CREATE TABLE movies (
  movie_id INTEGER PRIMARY KEY,
  title VARCHAR(160) NOT NULL,
  release_year INTEGER NOT NULL CHECK (release_year >= 1888),
  director_id INTEGER NOT NULL REFERENCES directors(director_id),
  UNIQUE (title, release_year)
```

```
);
CREATE TABLE genres (
  genre_id INTEGER PRIMARY KEY,
  name VARCHAR(60) NOT NULL UNIQUE
);
CREATE TABLE movie_genres (
  movie_id INTEGER NOT NULL REFERENCES movies(movie_id) ON DELETE CASCADE,
  genre_id INTEGER NOT NULL REFERENCES genres(genre_id) ON DELETE RESTRICT,
  PRIMARY KEY (movie_id, genre_id)
);
```

A2 - примерен отговор

```
ALTER TABLE movies ADD COLUMN runtime_min INTEGER CHECK (runtime_min > 0);
ALTER TABLE movies ADD CONSTRAINT uq_movies_title_year UNIQUE (title, release_year);
ALTER TABLE movies ADD CONSTRAINT fk_movies_director
  FOREIGN KEY (director_id) REFERENCES directors(director_id);
CREATE INDEX idx_movies_director_id ON movies(director_id);
```

A3 - структура

```
CREATE TABLE students (
  student_id INTEGER PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL
);
CREATE TABLE courses (
  course_id INTEGER PRIMARY KEY,
  title VARCHAR(160) NOT NULL UNIQUE
);
CREATE TABLE enrollments (
  student_id INTEGER NOT NULL REFERENCES students(student_id),
  course_id INTEGER NOT NULL REFERENCES courses(course_id),
  grade NUMERIC(3,2) CHECK (grade BETWEEN 2 AND 6),
  enrollment_date DATE NOT NULL,
  PRIMARY KEY (student_id, course_id)
);
```

A4 - 3НФ структура

```
Customers(customer_id PK, customer_name)
Products(product_id PK, product_name)
Orders(order_id PK, customer_id FK)
OrderItems(order_id FK, product_id FK, quantity,
           PK(order_id, product_id))
```

A5

```
movie_genres.movie_id REFERENCES movies(movie_id) ON DELETE CASCADE
movie_genres.genre_id REFERENCES genres(genre_id) ON DELETE RESTRICT
movies.director_id REFERENCES directors(director_id) ON DELETE RESTRICT
```

B1/B2

```
INSERT INTO directors(director_id, full_name) VALUES (1, 'Christopher Nolan');
INSERT INTO genres(genre_id, name) VALUES (1, 'Drama');
INSERT INTO movies(movie_id, title, release_year, director_id)
VALUES (1, 'Oppenheimer', 2023, 1);
INSERT INTO movie_genres(movie_id, genre_id) VALUES (1, 1);

UPDATE movies SET runtime_min = 180 WHERE title = 'Oppenheimer';
DELETE FROM movie_genres WHERE movie_id = 10 AND genre_id = 3;
```

C1/C2/C3/C4

```
-- C1
SELECT m.title, m.release_year, d.full_name
```

```
FROM movies m JOIN directors d ON d.director_id = m.director_id
WHERE m.release_year > 2020
ORDER BY m.release_year DESC;

-- C2
SELECT d.full_name, COUNT(*) AS movie_count
FROM directors d JOIN movies m ON m.director_id = d.director_id
GROUP BY d.director_id, d.full_name
HAVING COUNT(*) >= 2;

-- C3
SELECT DISTINCT title FROM movies WHERE release_year >= 2020;

-- C4
SELECT m.title, d.full_name
FROM movies m JOIN directors d ON d.director_id = m.director_id;
```

D1/D2 - проверка

```
D1: кандидат-ключ {StudentID, CourseID}.
3NF tables: Students(StudentID, StudentName), Teachers(TeacherID, TeacherName),
Courses(CourseID, CourseName, TeacherID), Enrollments(StudentID, CourseID, Grade).

D2: ключове {A,B} и {A,C}. 3НФ е изпълнена, защото B е първичен.
BCNF е нарушена от C -> B. BCNF декомпозиция: R1(C,B), R2(A,C).
```